

Bachelor Thesis

im Studiengang Geoinformatik

zur Erlangung des Grades eines
Bachelor of Science (B.Sc.)

über das Thema

LLM-gestützte Generierung von Playwright-E2E-Tests aus Use Cases

Einfluss von UI-Kontext am Beispiel von WebGIS-Anwendungen

von

Nils Große Beikel

Betreuer (Universität): Dr. Christian Knoth
Betreuer (Unternehmen): Dr. Thore Fechner
Matrikelnummer : 546548
Abgabedatum : 31. August 2026

Inhaltsverzeichnis

Abbildungsverzeichnis	IV
Tabellenverzeichnis	V
Abkürzungsverzeichnis	VI
1 Einleitung	1
1.1 Motivation und Problemstellung	1
1.2 Zielsetzung und Erkenntnisinteresse	1
1.3 Aufbau der Arbeit	1
2 Grundlagen	2
2.1 Large Language Models	2
2.1.1 Funktionsprinzip und Kontext als Qualitätsfaktor	2
2.1.2 Einsatz im Software Engineering	3
2.2 E2E-Testautomatisierung mit Playwright	4
2.2.1 Teststufen und Einordnung von E2E-Tests	4
2.2.2 Playwright: Selektoren, Assertions, asynchrone UIs	5
2.3 Open Pioneer Trails	6
2.3.1 Framework-Überblick und Komponentenmodell	6
2.3.2 data-testid-Konventionen und Testbarkeit	8
2.4 WebGIS-Anwendungen und Testkomplexität	8
3 Stand der Forschung	10
3.1 LLM-gestützte Testgenerierung	10
3.2 Forschungslücke und Einordnung dieser Arbeit	11
4 Methodik und Forschungskonzept	13
4.1 Überblick und Forschungsdesign	13
4.2 Use Cases: Definition und Format	15
4.3 Versuchsaufbau: Die vier Kontextstufen	16
4.3.1 Stufe 1 - Baseline: kein UI-Kontext	16
4.3.2 Stufe 2 - Automatischer Accessibility-Snapshot	16
4.3.3 Stufe 3 - Automatisch generierte UI-Map	16
4.3.4 Stufe 4 - Manuell erstellte UI-Map	16
4.3.5 Bonus-Stufe 5 - Self-Improvement-Loop	16
4.4 LLM-Konfiguration (Modell, Temperatur, Prompt-Struktur)	16

4.5	Bewertungskriterien	16
5	Implementierung der Evaluationsumgebung	17
5.1	Demo-WebGIS-Anwendung	17
5.2	Use-Case-Sammlung	18
5.3	Manuell erstellte Referenztests	19
6	Durchführung und Evaluation	23
6.1	LLM-gestützter Generierungsworkflow	23
6.2	Ergebnisse je Kontextstufe	23
6.3	Vergleich der Kontextstufen	23
6.4	Vergleich mit manuell erstellten Tests	23
6.5	GIS-spezifische Besonderheiten	23
7	Diskussion	24
7.1	Interpretation der Ergebnisse	24
7.2	Limitationen	24
7.3	Handlungsempfehlungen	24
8	Fazit und Ausblick	24
	Anhang	25
	Literaturverzeichnis	26

Abbildungsverzeichnis

Abbildung 1: Testpyramide nach Mike Cohn	5
Abbildung 2: Open Pioneer Trails Paketökosystem	7
Abbildung 3: Generierungs- und Evaluationspipeline	14
Abbildung 4: Ausprägungen der Variable UI-Kontext	15
Abbildung 5: Self-Improvement-Loop (Stufe 5)	16
Abbildung 6: Demo-WebGIS-Anwendung	17

Tabellenverzeichnis

Tabelle 1: Übersicht der Use Cases mit Komplexität und getesteter Kernfunktion . . .	19
--	----

Abkürzungsverzeichnis

API	Application Programming Interface
ARIA	Accessible Rich Internet Applications
CSS	Cascading Style Sheets
DOM	Document Object Model
DWD	Deutscher Wetterdienst
E2E	End-to-End
EPSG	European Petroleum Survey Group
GIS	Geoinformationssystem
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
OPT	Open Pioneer Trails
PDF	Portable Document Format
PNG	Portable Network Graphics
PNPM	Performant Node Package Manager
UI	User Interface
WGS84	World Geodetic System 1984
WMS	Web Map Service

1 Einleitung

1.1 Motivation und Problemstellung

1.2 Zielsetzung und Erkenntnisinteresse

1.3 Aufbau der Arbeit

2 Grundlagen

2.1 Large Language Models

Large Language Models (LLMs) sind maschinelle Lernmodelle, die auf sehr großen Textmengen trainiert werden, um natürliche Sprache zu verarbeiten und zu erzeugen. Das Attribut „large“ bezieht sich dabei sowohl auf den Umfang der Trainingsdaten als auch auf die Anzahl der Modellparameter, die häufig im Bereich mehrerer Milliarden liegt. Technisch beruhen sie auf künstlichen neuronalen Netzen, genauer gesagt auf der Transformer-Architektur *Hou et al., 2024*. Der folgende Abschnitt erläutert zunächst das Funktionsprinzip und die Rolle des bereitgestellten Kontexts, bevor der Einsatz im Software Engineering betrachtet wird.

2.1.1 Funktionsprinzip und Kontext als Qualitätsfaktor

Moderne LLMs basieren auf der Transformer-Architektur, die von *Vaswani et al. (2017)* eingeführt wurde. Ihr zentraler Bestandteil ist der Self-Attention-Mechanismus. Er erlaubt es dem Modell, die Beziehungen zwischen allen Elementen einer Eingabe unabhängig von ihrer Position zu gewichten. Anders als bei früheren rekurrenten Ansätzen wird die Eingabe dadurch nicht sequentiell, sondern weitgehend parallel verarbeitet, was das Training auf sehr großen Textmengen erst praktikabel gemacht hat *ebd.* Vor der Verarbeitung wird der Text in sogenannte Tokens zerlegt. Tokens sind in der Regel keine ganzen Wörter, sondern Teilwörter oder einzelne Zeichen, auf denen das Modell dann ausschließlich arbeitet.

Die Texterzeugung erfolgt autoregressiv. Auf Basis der bereits vorliegenden Tokens berechnet das Modell für das nächste Token eine Wahrscheinlichkeitsverteilung über das gesamte Vokabular. Aus dieser Verteilung wird ein Token ausgewählt und an die Eingabe angehängt. Der Vorgang wiederholt sich, bis die Ausgabe abgeschlossen ist. Wie stark die Auswahl vom jeweils wahrscheinlichsten Token abweichen darf, steuert unter anderem der Temperatur-Parameter. Ein niedriger Wert führt zu deterministischeren, ein hoher zu variableren Ausgaben. Das ist für diese Arbeit relevant, denn nur eine über alle Versuche konstant gehaltene Konfiguration erlaubt einen aussagekräftigen Vergleich der Generierungsergebnisse *ebd.*

Alle Tokens, die das Modell für eine Generierung berücksichtigt, befinden sich im sogenannten Kontextfenster. Dieses umfasst die Eingabe (den Prompt) und die bereits erzeugte Ausgabe und ist in seiner Größe begrenzt. Wissen, das nicht in den

Modellgewichten verankert ist, also nicht während des Trainings gelernt wurde, muss dem Modell zur Laufzeit über dieses Fenster bereitgestellt werden *Brown et al., 2020*.

Eng damit verbunden ist die Fähigkeit zum In-Context Learning. *Brown et al. (ebd.)* zeigten, dass große Sprachmodelle Aufgaben allein anhand von Anweisungen oder Beispielen im Prompt lösen können, ohne dass die Modellgewichte angepasst werden. Das Modell „lernt“ hierbei nicht im klassischen Sinne, sondern nutzt die im Kontext bereitgestellten Informationen direkt als Grundlage für seine Vorhersage. Welche Inhalte im Prompt stehen, bestimmt damit maßgeblich die Ausgabe.

Daraus folgt ein zentraler Punkt für diese Arbeit. Ein LLM hat kein konkretes Wissen über eine individuell entwickelte Benutzeroberfläche. Informationen über die tatsächlich vorhandenen UI-Elemente, deren Selektoren und Zustände stehen weder in den Trainingsdaten noch sind sie zur Laufzeit zugänglich, solange sie nicht explizit im Prompt bereitgestellt werden. Die Qualität des generierten Testcodes hängt damit unmittelbar von Qualität und Umfang des bereitgestellten Kontexts ab.

2.1.2 Einsatz im Software Engineering

LLMs werden zunehmend im Software Engineering eingesetzt, insbesondere zur Generierung von Quellcode. Sichtbar wird das an Werkzeugen wie GitHub Copilot, das auf Grundlage des umgebenden Quellcode-Kontexts Vorschläge zur Codevervollständigung erzeugt und direkt in die Entwicklungsumgebung integriert ist. Der systematische Literaturüberblick von *Hou et al. (2024)* zeigt, dass der Einsatz von LLMs im Software Engineering inzwischen breit erforscht wird und deutlich über die reine Codevervollständigung hinausreicht.

Ein Anwendungsfeld ist die automatisierte Testgenerierung. LLMs werden hier eingesetzt, um aus natürlichsprachlichen oder strukturierten Beschreibungen Unit-Tests, Akzeptanztests und End-to-End (E2E)-Tests zu erzeugen. Bestehende Ansätze beziehen sich überwiegend auf generische Webanwendungen *Celik, Mahmoud, 2025; Ferreira et al., 2025*. GIS-spezifische Anwendungen werden aktuell kaum berücksichtigt. Eine ausführliche Betrachtung des Forschungsstands folgt in Kapitel 3.

Diesem Potential stehen bekannte Grenzen gegenüber. Da LLMs ihre Ausgaben auf Basis von Wahrscheinlichkeiten erzeugen, können sie plausibel wirkende, faktisch aber nicht existierende Inhalte produzieren, bei der Testgenerierung etwa erfundene Selektoren oder Aufrufe nicht vorhandener Funktionen. Dieses Phänomen wird als Halluzination bezeichnet *Hou et al., 2024*. Hinzu kommt das fehlende Wissen über die konkrete Anwendung und

der Umstand, dass das Modell keinen Zugriff auf den tatsächlichen Laufzeitzustand der Oberfläche hat. Gerade für E2E-Tests, die gezielt auf konkrete User Interface (UI)-Elemente zugreifen müssen, sind diese Einschränkungen besonders relevant.

Grundsätzlich lassen sich Sprachmodelle auf zwei Wegen an eine spezifische Aufgabe anpassen: durch Fine-Tuning, also das Nachtrainieren der Modellgewichte auf aufgabenspezifischen Daten, oder durch Prompting, bei dem das Verhalten allein über die Gestaltung der Eingabe gesteuert wird. Prompting beruht auf dem in Abschnitt 2.1.1 beschriebenen In-Context Learning *Brown et al., 2020*. Diese Arbeit verfolgt durchgängig den Prompting-Ansatz, da er gegenüber Fine-Tuning sowohl einen geringeren Ressourcenaufwand erfordert als auch eine bessere Generalisierbarkeit bietet. Ein prompt-basierter Workflow lässt sich ohne erneutes Training auf andere Anwendungen und Use Cases übertragen, während ein feinjustiertes Modell weitgehend auf seinen Trainingskontext beschränkt bliebe.

2.2 E2E-Testautomatisierung mit Playwright

Softwaretests lassen sich nach ihrem Umfang und ihrer Nähe zum Nutzerverhalten in verschiedene Stufen einteilen. Im Folgenden werden E2E-Tests in diese Stufen eingeordnet und anschließend das in dieser Arbeit verwendete Testframework Playwright vorgestellt.

2.2.1 Teststufen und Einordnung von E2E-Tests

Eine verbreitete Einteilung von Softwaretests ist die Testpyramide, die von *Poenisch (2022)* beschrieben wird. Sie unterscheidet drei Ebenen, wie in Abbildung 5 dargestellt. Ganz unten stehen Unit-Tests, die einzelne Funktionen oder Komponenten prüfen. Sie laufen schnell und werden in großer Anzahl geschrieben. Darüber liegen die Integrationstests, die das Zusammenspiel mehrerer Komponenten betrachten. An der Spitze stehen E2E-Tests, die die Anwendung als Ganzes aus Sicht der Nutzer prüfen.

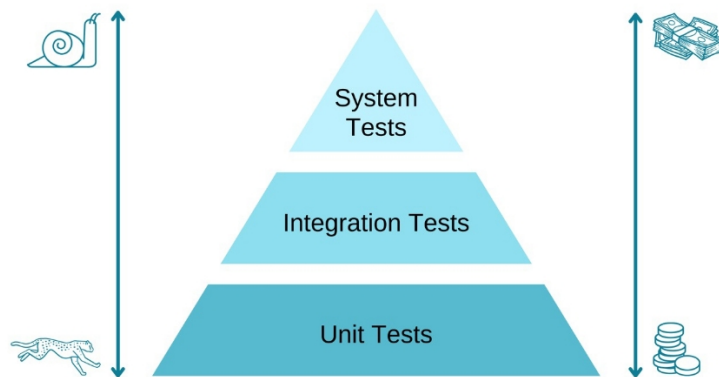


Abbildung 1: Die Testpyramide nach Mike Cohn (Quelle: *Poenisch*, 2022)

Ein E2E-Test steuert dazu einen Browser und führt dieselben Schritte aus wie ein Mensch. Dazu gehören auf Schaltflächen klicken, Felder ausfüllen und durch die Oberfläche navigieren. Geprüft wird, ob am Ende das erwartete Ergebnis eintritt. Dadurch geben E2E-Tests eine gute Auskunft darüber, ob eine fachliche Anforderung tatsächlich umgesetzt ist. Allerdings sind sie langsam, aufwendig zu erstellen und vergleichsweise anfällig und können schon bei kleinen Änderungen an der UI fehlschlagen. Deshalb wird empfohlen, ihre Anzahl, entsprechend der Form der Pyramide, gering zu halten.

Für WebGIS-Anwendungen sind E2E-Tests besonders relevant. Viele Probleme zeigen sich erst im Zusammenspiel der Komponenten und Interaktionen. Dazu gehört das asynchrone Laden von Geodaten, das Rendern der Karte oder die Auswirkungen der Layersteuerung auf die Kartenanzeige. Solche Abläufe lassen sich mit Unit-Tests kaum abbilden, weil sie den vollständigen, im Browser laufenden Zustand voraussetzen.

2.2.2 Playwright: Selektoren, Assertions, asynchrone UIs

Playwright ist ein Open Source E2E-Test-Framework von Microsoft zur Automatisierung von Browsern. Es steuert die Browser-Engines Chromium, Firefox und WebKit über eine einheitliche Programmierschnittstelle und führt Tests in einem echten Browser aus. Die Tests werden in dieser Arbeit in TypeScript geschrieben, das Playwright unterstützt *Microsoft*, 2026.

Um mit der Benutzeroberfläche zu interagieren, muss ein Test die jeweiligen Elemente finden. Playwright nutzt dafür sogenannte Lokatoren. Empfohlen wird, Elemente über nutzerseitige Eigenschaften anzusprechen, etwa über ihre ARIA-Rolle oder ihren

sichtbaren Text (`getByText`, `getByRole`). Solche Selektoren sind robuster gegenüber Änderungen am Markup als Cascading Style Sheets (CSS)- oder XPath-Ausdrücke, die an die konkrete Document Object Model (DOM)-Struktur gebunden sind. Eine Alternative sind Test-IDs. Über das Attribut `data-testid` lässt sich ein Element gezielt für Tests markieren und mit `getByTestId` ansprechen, unabhängig von Darstellung und Position. Welches Attribut als Test-ID gilt, ist konfigurierbar *Microsoft, 2026*.

Ein wesentliches Merkmal von Playwright ist das automatische Warten. Vor jeder Aktion prüft das Framework, ob das Zielelement sichtbar, stabil und bedienbar ist, ansonsten wird bis zu einem Timeout gewartet. Dadurch müssen nicht für jede Interaktion manuelle Wartezeiten eingebaut werden, was eine häufige Ursache für instabile Tests beseitigt. Dasselbe Prinzip gilt für Assertions. Mit `expect` formulierte Zusicherungen wie `expect(locator).toBeVisible()` oder `toHaveText()` werden so lange wiederholt, bis die Bedingung erfüllt ist oder die Zeit abläuft. Eine Assertion prüft damit nicht einen Momentzustand, sondern einen Zustand, der sich auch verzögert einstellen darf.

Für Fälle, die das automatische Warten nicht abdeckt, stellt Playwright extra Wartemethoden bereit, etwa auf eine bestimmte Netzwerkantwort (`waitForResponse`) oder einen Ladezustand der Seite (`waitForLoadState`). Das ist gerade bei WebGIS-Anwendungen wichtig, in denen Kartenkacheln und Geodaten asynchron über das Netz geladen werden und erst danach sichtbar auftauchen. Ein Test, der zu früh prüft, würde fehlschlagen, obwohl die Anwendung korrekt arbeitet. Gerade beim asynchronen Laden von Kartenkacheln und Geodaten greift damit beides ineinander: das automatische Warten für DOM-Elemente und gezielte Wartepunkte auf Netzwerkebene.

2.3 Open Pioneer Trails

2.3.1 Framework-Überblick und Komponentenmodell

Open Pioneer Trails ist ein Open Source Framework zur Entwicklung webbasierter GeoIT-Anwendungen. Es ist als gemeinsames Projekt der con terra und 52°North im Rahmen der Open-Pioneer-Initiative entstanden und steht unter der Apache 2.0 Lizenz frei auf GitHub zur Verfügung. Trails ist ein Entwicklungsframework und kein fertiges Produkt. Anwendungen werden also nicht konfiguriert, sondern programmiert, wobei das Framework wiederverwendbare Komponenten und Vorlagen bereitstellt *52°North GmbH, Matthes Rieke Managing Director, 2026*.

Technisch besteht Trails aus modernen Webtechnologien. Die Anwendungen werden in TypeScript mit React geschrieben, das Styling basiert auf Chakra UI, als Kartenkomponente

wird OpenLayers verwendet, die Paketverwaltung läuft über Performant Node Package Manager (PNPM) und als Build-System wird Vite verwendet *Open Pioneer*, 2026.

Eine Open Pioneer Trails-Anwendung ist auf Paketen aufgebaut. Ein Paket bündelt React-Komponenten, Services, Übersetzungen und Styling. Die Anwendung selbst ist ebenfalls ein Paket und wird zu einer Web Component gebaut, dadurch kann sie einfach in bestehende Web-Anwendungen integriert werden. Aktuell werden zwei Gruppen wiederverwendbarer Pakete bereitgestellt. Die Core Packages enthalten kartenlose Basisfunktionen wie Authentifizierung, Notifier, Internationalisierung mit I18n und einen Hypertext Transfer Protocol (HTTP)-Service. Die OpenLayers Base Packages bauen darauf auf und liefern die Kartenkomponenten, darunter Layerübersicht, Kartennavigations-Werkzeuge, Legende, Selektion und Ergebnisliste, Suche, Messwerkzeug, Koordinatenanzeige und Kartendruck. Abbildung 2 gibt einen Überblick über das Paketökosystem.

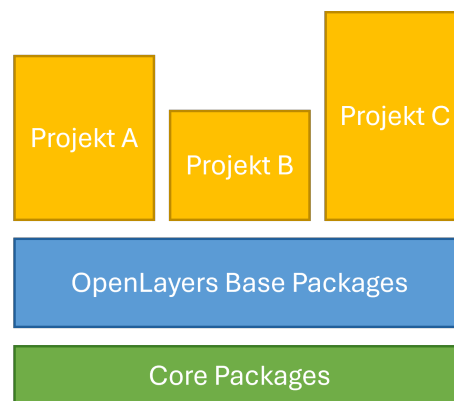


Abbildung 2: Das Open Pioneer Trails Paketökosystem

Die Anbindung der Karte erfolgt nicht direkt über OpenLayers, sondern über eine eigene Schnittstelle. Diese erweitert das Datenmodell, vereinfacht den Umgang mit dem Kartenzustand und stellt ihn als TypeScript-Schnittstelle (MapModel) bereit. Der direkte Zugriff auf die OpenLayers-Application Programming Interface (API) bleibt bei Bedarf möglich *52°North GmbH, Arne Vogt*, 2025. Für die Demo-Anwendung relevant sind unter anderem die Pakete für Karte, Layerübersicht, Basemap-Auswahl, Suche, Messen und Druck.

Trails richtet sich explizit an Entwickler und erfordert Programmierkenntnisse. Dafür lässt es Layout und Funktionalität frei gestalten und bringt typische Geoinformationssystem (GIS)-Widgets als fertige Komponenten mit. Für die Erstellung einer spezifischen Demo-WebGIS-Anwendung im Rahmen dieser Arbeit ist es damit eine passende Grundlage.

2.3.2 data-testid-Konventionen und Testbarkeit

Ein verbreitetes Mittel, um Elemente in Tests gezielt anzusprechen, ist das Hypertext Markup Language (HTML)-Attribut `data-testid`. Es wird ausschließlich für Tests vergeben und ist von Styling und DOM-Struktur unabhängig. Ein Test, der ein Element über seine Test-ID anspricht, bleibt auch dann stabil, wenn sich CSS-Klassen oder Strukturen ändern. Playwright kann dann wie in Abschnitt 2.2.2 beschrieben mit `getByTestId` darauf zugreifen.

Trails unterstützt diese Konvention standardmäßig. Alle öffentlichen Komponenten teilen sich die gemeinsame Schnittstelle `CommonComponentProps`, die optional eine `data-testid` entgegennimmt und an das DOM weiterreicht. Bei allen anderen Elementen von Chakra UI werden nur Chakra-bezogene Style-Props herausgefiltert und alles andere ans DOM übergeben, darunter auch die Test-ID *Chakra Systems*, 2026. Den Wert gibt dabei die Entwicklung vor, einen automatischen Standardwert vergibt das Framework nicht. Für gut testbare Anwendungen müssen die `data-testid`-Attribute daher bewusst und sinnvoll im Anwendungscode gesetzt werden.

Neben den Test-IDs sind die Komponenten auf Barrierefreiheit ausgelegt und folgen Accessible Rich Internet Applications (ARIA)-Konventionen *52°North GmbH, Arne Vogt*, 2025. Elemente lassen sich dadurch beispielsweise über ihre Rolle oder ihren sichtbaren Textinhalt finden. Playwright kann anhand von Rolle und Bezeichnung (`getByRole/getByText`) auf diese Elemente zugreifen. Damit bietet Trails zwei robuste Wege, ein Element zu adressieren.

2.4 WebGIS-Anwendungen und Testkomplexität

WebGIS-Anwendungen stellen die Testautomatisierung vor Herausforderungen, die generische Webanwendungen in dieser Form nicht haben. Der Grund dafür ist, wie Karten und Geodaten im Browser dargestellt und geladen werden. Ein zentrales Problem ist die Darstellung der Karte selbst. OpenLayers zeichnet die Karte auf ein Canvas-Element, also als Pixelgrafik und nicht als einzelne DOM-Elemente *OpenLayers (Institution)*, 2026. Ein Locator kann zwar das Canvas-Element finden, aber nicht die Inhalte darin, z.B. ob ein bestimmter Layer gerade sichtbar ist. Was in der Karte dargestellt wird, ist also nicht direkt über das DOM zugänglich. Zu testen, ob ein Feature auf der Karte sichtbar ist, lässt sich daher nicht auf demselben Wege prüfen wie die Sichtbarkeit einer Schaltfläche.

Hinzu kommt, dass Geodaten asynchron über das Netz geladen werden. Kartenkacheln, Web Map Service (WMS)-Layer, GetFeatureInfo-Abfragen oder Anfragen des Geocoders

treffen mit unterschiedlicher Verzögerung ein. Ein Test muss abwarten, bis die Daten geladen und gerendert wurden, bevor getestet werden kann. Das in Abschnitt 2.2.2 beschriebene automatische Warten von Playwright hilft hier nur bedingt, da es sich auf die Bedienbarkeit von DOM-Elementen bezieht. Ob Kartenkacheln vollständig dargestellt werden, lässt sich darüber nicht erkennen, sodass auf andere Methoden wie das Abwarten einer Netzwerkantwort zurückgegriffen werden muss.

Die Oberfläche einer WebGIS-Anwendung ist außerdem stark vom aktuellen Zustand abhängig. Ein Layer kann sichtbar oder unsichtbar sein, die Legende bezieht sich auf sichtbare Layer und eine Informationsanzeige erscheint erst, nachdem ein Feature in der Karte angeklickt wurde. Auch hier ist das Verhalten von Interaktionen wichtig. Die Informationsanzeige bei Klick auf ein Feature kann zum Beispiel deaktiviert sein, wenn gerade eine Messfunktion zum Messen einer Linie aktiv ist. Dieselbe Interaktion kann je nach Zustand zu einem anderen Ergebnis führen, weshalb die Vorbedingungen eines Tests genau definiert sein müssen.

Außerdem sind viele Interaktionen abhängig von Koordinaten. Angenommen der Nutzer klickt auf eine Position, auf eine Markierung oder zeichnet durch mehrere Klicks eine Linie. Der Test muss dafür auf Pixelkoordinaten im Canvas klicken statt auf ein bekanntes Element. Das Ergebnis hängt dann von Kartenausschnitt, Zoomstufe und Projektion ab. Ein Klick auf eine beliebige Position ist dabei etwas anderes als das gezielte Treffen eines bestimmten Features, das an einer bekannten Stelle liegt.

3 Stand der Forschung

3.1 LLM-gestützte Testgenerierung

Die automatisierte Testgenerierung mit LLMs ist ein Teilgebiet des in Kapitel 2.1.2 beschriebenen Einsatzes von LLMs im Software Engineering (*Hou et al., 2024*). *Celik & Mahmoud (2025)* systematisieren die bestehenden Ansätze in einer Literaturübersicht und unterscheiden zwischen vier methodischen Kategorien: prompt-basierte Verfahren, feedback-getriebene Verfahren, Fine-Tuning und hybride Kombinationen. Der überwiegende Teil der Literatur befasst sich mit der Generierung von Unit-Tests aus vorhandenem Quellcode (ebd.). Diese Arbeit verfolgt einen anderen Ansatz: Der Test soll aus einer fachlichen Anforderung entstehen, ohne dass der Quellcode der Anwendung als direkte Eingabe dient.

Dass natürlichsprachliche Beschreibungen als alleinige fachliche Eingabe ausreichen können, zeigen *Plein et al. (2023)* in einer Machbarkeitsstudie. Sie untersuchen, ob LLMs aus natürlichsprachlichen Bug Reports ausführbare Testfälle erzeugen können. Für einen relevanten Teil der Bug Reports gelingt die Generierung fehlerproduzierender Tests. Zugleich treten typische Schwächen auf, etwa unvollständige oder nicht kompilierbare Ausgaben (ebd.). Die Studie bleibt allerdings auf der Unit-Test-Ebene und ohne Bezug zu einer Benutzeroberfläche, weshalb sich die Ergebnisse nicht direkt auf E2E-Tests übertragen lassen.

Für den E2E-Testbereich liegt eine Fallstudie von *Ferreira et al. (2025)* vor. Der dort vorgestellte Ansatz arbeitet zweistufig: Ein LLM erzeugt zunächst aus User Stories Testszenarien im Gherkin-Format und generiert daraus anschließend ausführbare Testskripte für das Testframework Cypress. Als Kontext erhält das Modell neben der User Story auch den HTML-Code der betroffenen Seiten, der UI-Kontext wird also in fester Form mitgeliefert. Der Großteil der generierten Szenarien (95 %) und Skripte (92 %) wurde als brauchbar bewertet; von den Skripten waren 60 % direkt einsetzbar, weitere 8 % nach kleineren Korrekturen (ebd.). Zudem zeigte sich, dass die semantische Relevanz der generierten Tests von 60 % auf 92 % stieg, wenn der bereitgestellte Kontext nachträglich mitgeliefert wurde. Fehlender Kontext war zugleich die häufigste Fehlerursache. Dieser Zusammenhang zwischen Kontextqualität und Testqualität wurde jedoch nur beobachtet und nicht als kontrollierte Variable systematisch gemessen. Außerdem ist zu beachten, dass die Zielanwendungen generische Webanwendungen mit formular- und navigationsbasierten Interaktionen sind und keine karten- oder canvas-basierten Oberflächen beinhalten.

Einen anderen Weg zur Kontextgewinnung wählen *Alian et al. (2024)* mit AutoE2E. Der Ansatz identifiziert Features einer Webanwendung automatisch und leitet daraus vollständige E2E-Tests ab. Der Kontext stammt hier nicht aus einer Anforderungsbeschreibung, sondern wird durch Erkundung der Anwendung selbst gewonnen, indem der UI-Zustand systematisch gecrawlt wird. AutoE2E erreicht eine höhere Feature-Abdeckung als bisherige Verfahren und agentenbasierte Baselines und bildet damit den aktuellen Stand der LLM-gestützten E2E-Testgenerierung für Webanwendungen ab (ebd.). Von dem Ansatz dieser Arbeit unterscheidet sich AutoE2E in zwei Punkten: Die Tests entstehen aus dem beobachteten Anwendungszustand statt aus fachlichen Use Cases, und die Form des UI-Kontexts, den das LLM erhält, ist durch die Architektur fest vorgegeben und ändert sich nicht.

Die betrachteten Arbeiten belegen gemeinsam, dass die Generierung ausführbarer Tests für Webanwendungen aus natürlichsprachlichen Anforderungen machbar ist (*Plein et al., 2023; Ferreira et al., 2025; Alian et al., 2024*). Alle Ansätze nutzen dabei eine Form von Anwendungs- oder UI-Kontext, sei es der HTML-Code der Seiten oder der gecrawlte Anwendungszustand. Allerdings untersucht keine der Arbeiten, wie sich Form und Umfang des bereitgestellten UI-Kontexts auf die Qualität der generierten Tests auswirken.

3.2 Forschungslücke und Einordnung dieser Arbeit

Für WebGIS-Anwendungen mit canvas-basiertem Kartenrendering existieren nach eigener Recherche keine wissenschaftlichen Arbeiten zur LLM-gestützten Testgenerierung. Die in Kapitel 3.1 betrachteten Ansätze setzen durchgängig DOM-basierte Oberflächen voraus, deren Elemente über Selektoren ansprechbar sind. Diese Annahme trifft aber für einige Bestandteile von WebGIS-Anwendungen nicht zu. Auch für das Framework Open Pioneer Trails liegen keine wissenschaftlichen Untersuchungen vor. Ob und wie sich die Ergebnisse auf diesen Anwendungsfall übertragen lassen, ist damit offen.

Die zweite Lücke betrifft den UI-Kontext selbst. Dass der bereitgestellte UI-Kontext Einfluss auf die Ausgabequalität eines LLM hat, ist auf konzeptioneller Ebene zu erklären (In-Context Learning, vgl. Kapitel 2.1.1). In der Literatur zur E2E-Testgenerierung fehlt eine Untersuchung, die diesen Einfluss als einzige unabhängige Variable untersucht und alle übrigen Faktoren konstant hält.

Diese Arbeit setzt an beiden Lücken an. Sie schließt an den Workflow von der Anforderung zum Test an (*Plein et al., 2023; Ferreira et al., 2025*) und grenzt sich damit von editorbasierten Werkzeugen ab, die aus dem umgebenden Quellcode

generieren. Ihr Beitrag lässt sich in drei Punkte zusammenfassen. Erstens liefert sie erste Untersuchungen zu LLM-gestützter E2E-Testgenerierung für WebGIS-Anwendungen, einschließlich Interaktionen mit der Karte. Zweitens wird der Einfluss des UI-Kontexts als einzige unabhängige Variable untersucht, und drittens wird der für DOM-basierte Tests nicht sichtbare Kartenzustand prüfbar gemacht.

4 Methodik und Forschungskonzept

4.1 Überblick und Forschungsdesign

Im Folgenden werden die Methodik und das Forschungskonzept zur Beantwortung der Forschungsfrage dargestellt, welchen Einfluss der UI-Kontext auf die Qualität LLM-generierter Playwright-E2E-Tests für Open Pioneer Trails (OPT)-basierte WebGIS-Anwendungen hat. Die in der Einleitung formulierten Teilfragen werden dabei mit in das Untersuchungsdesign einbezogen. Bei der Methodik handelt es sich um einen experimentellen, vergleichenden Aufbau in einer kontrollierten Demo-Umgebung.

Der UI-Kontext ist die einzige unabhängige Variable. Alle anderen Faktoren wie Use Cases, LLM und dessen Konfiguration, OPT-Playwright-Skill sowie die manuell erstellten Referenztests werden konstant gehalten. Nur unter diesen Bedingungen sind die Qualitätsunterschiede der generierten Tests dem Kontext zurechenbar und werden nicht durch andere Einflüsse verfälscht. Dieses Vorgehen knüpft an das in Kapitel 2.1.1 beschriebene In-Context Learning an, bei dem der Kontext im Prompt die Ausgabe maßgeblich bestimmt.

Von der unabhängigen Variable abzugrenzen sind die technischen Eigenschaften, die die Anwendung selbst zur Überprüfbarkeit bereitstellt. Während der UI-Kontext beschreibt, was dem LLM im Prompt mitgeteilt wird, bezeichnen Maßnahmen wie die Vergabe von data-testid-Attributen und der Zugriff auf das Kartenmodell über `__openPioneerMap` (siehe Kapitel 5), was die Anwendung unabhängig vom Prompt für Tests bereitstellt. Diese Möglichkeiten bestehen bereits ab Stufe 1 und bleiben über alle vier Stufen unverändert. Variabel ist nur, ob und in welcher Form dem LLM im Prompt mitgeteilt wird, dass sie existieren. Die Bereitstellung dieser technischen Möglichkeiten ist damit keine eigene Stufe des Experiments, sondern eine über alle Stufen geltende Voraussetzung, ohne die der UI-Kontext nicht die einzige unabhängige Variable wäre.

Generiert werden die Tests aus den fachlichen Anforderungen des jeweiligen Use Case und nicht aus dem Quellcode der Anwendung. Diese Abgrenzung ist wichtig im Hinblick auf editorbasierte Werkzeuge wie GitHub Copilot, die direkt aus dem umgebenden Code-Kontext Antworten generieren (siehe Kapitel 2.1.2).

Die Struktur des Prompts wird dabei über alle vier Stufen konstant gehalten. Es ändert sich nur der enthaltene UI-Kontext. Verschiedene Prompting-Strategien sind nicht Teil dieser Untersuchung.

Als kontrollierte Konstante im Prompt wird ein OPT-Playwright-Skill verwendet. Als OPT-Playwright-Skill wird im Rahmen dieser Arbeit eine feste, appunabhängige Anleitung bezeichnet, die dem LLM bei jeder Generierung als Teil des Prompts mitgegeben wird. Sie legt allgemeine Umgangsweisen für den erzeugten Testcode fest, darunter die Verwendung von React mit TypeScript, den Umgang mit `async`, `await`, OPT-spezifische Besonderheiten sowie das erwartete Ausgabeformat. App-spezifische Selektoren enthält der Skill bewusst nicht, damit es hier zu keiner Vermischung mit dem UI-Kontext kommt, sondern unabhängig davon über alle vier Stufen unverändert angewendet wird. Die genaue Ausgestaltung des Skills wird in Kapitel 4.4 beschrieben.

Die vier Stufen bilden einen Verlauf von minimalem zu maximalem UI-Kontext. Stufe 1 gibt dem LLM keine Informationen über die Oberfläche, während die höheren Stufen den Kontext schrittweise erweitern und strukturieren.

Abbildung zeigt den Ablauf eines Generierungsvorgangs. Use Case, OPT-Playwright-Skill und der UI-Kontext der jeweiligen Stufen bilden zusammen den Prompt für das LLM. Das LLM erzeugt daraus einen Playwright-Test, der ausgeführt und anhand der Kriterien aus Kapitel 4.5 bewertet wird. Die manuellen Referenztests aus Kapitel 5.3 dienen dabei als Vergleichsmaßstab.

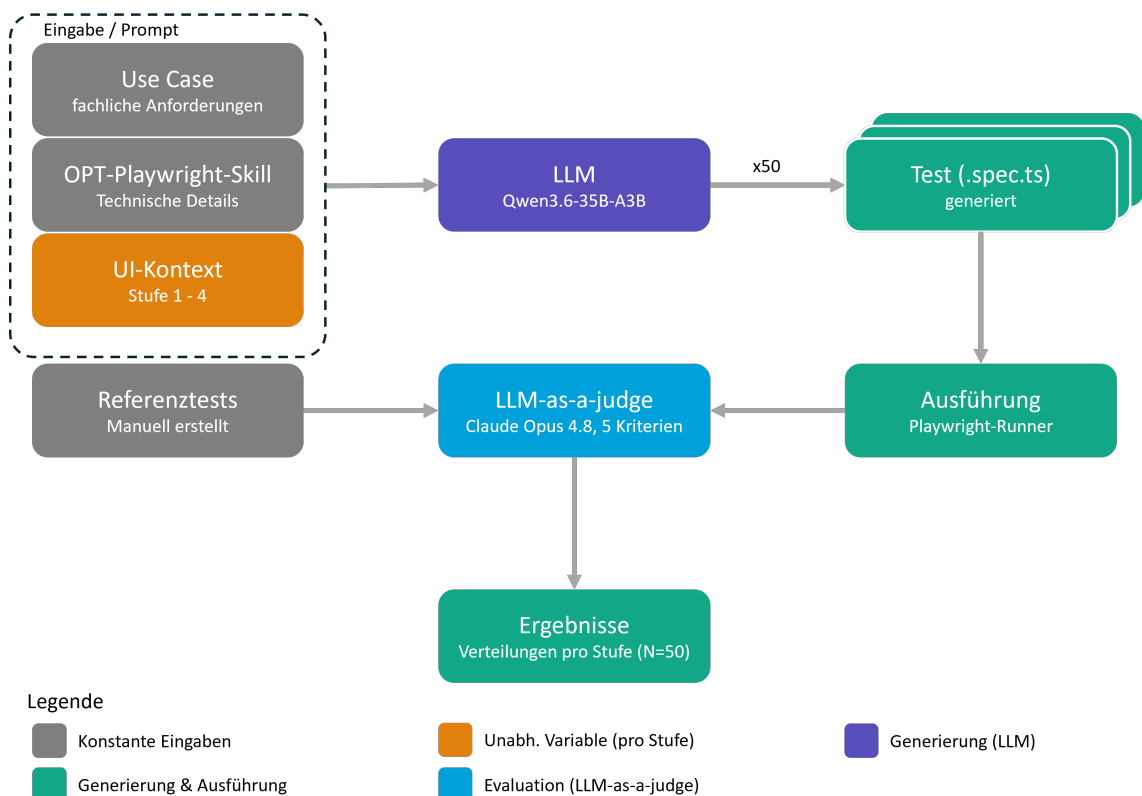


Abbildung 3: Generierungs- und Evaluationspipeline. Der UI-Kontext variiert als unabhängige Variable (Stufen 1–4), alle übrigen Eingaben sind konstant

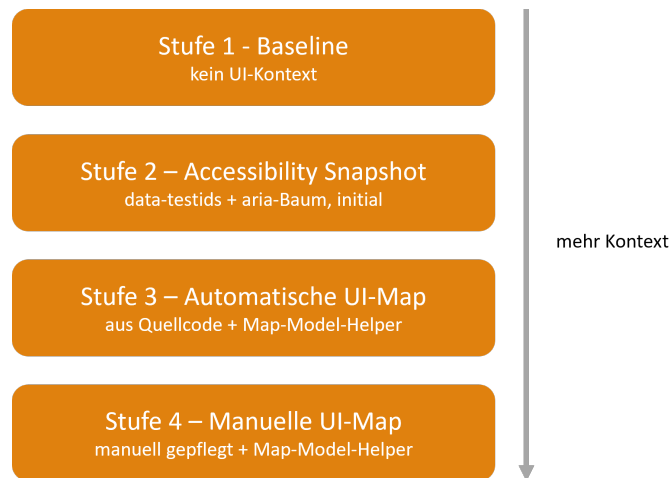


Abbildung 4: Die vier Ausprägungen der Variable UI-Kontext mit steigendem Informationsgehalt

4.2 Use Cases: Definition und Format

Die Use Cases sind die fachliche Eingabe, aus der die Tests generiert werden. Sie sind über alle vier Stufen identisch und gehören damit zu den konstant gehaltenen Faktoren.

Jeder Use Case beschreibt eine Nutzerinteraktion mit der WebGIS-Anwendung auf fachlicher Ebene aus Sicht des Anwenders. Die Schritte enthalten bewusst keine Selektoren oder andere technischen Details der Oberfläche, damit das Wissen allein aus dem UI-Kontext der jeweiligen Stufe kommt. Zudem macht es keinen Sinn in einer Nutzerinteraktion technische Details der Anwendung festzuhalten, da dies auch keine Voraussetzung der Anwender ist.

Die Use Cases werden im Markdown-Format erfasst. So sind sie für LLMs einfach zu verarbeiten und folgen einer einheitlichen Struktur, die für alle Stufen gleich verwendet wird. Jeder Use Case besteht aus Informationen zu `id`, `title`, `description`, `preconditions`, `steps`, `expected results` und `complexity`. Titel und Beschreibung geben an welche Interaktion der Use Case abbildet. Die Vorbedingungen halten fest, was vor der Ausführung erfüllt sein muss. Die Schritte beschreiben die durchzuführende Interaktion des Anwenders und die erwarteten Ergebnisse legen fest, welcher Zustand danach eintreten soll.

Die Eigenschaft `complexity` gruppiert die Use Cases in drei Schwierigkeitsstufen: `easy`, `medium`, `hard`. Die Einstufung basiert nicht allein auf der Anzahl der Schritte, sondern vor allem nach der Art und Tiefe der Interaktionen. Einfache Use Cases prüfen einzelne Schaltflächen oder Sichtbarkeiten, komplexere fordern hingegen asynchrone Prozesse,

Karteninteraktionen oder das Zusammenspiel mehrerer Komponenten. Die Abstufung stellt sicher, dass bei der Untersuchung verschiedene Komplexitäten typischer WebGIS-Interaktionen abgedeckt und nicht nur einfache Fälle betrachtet werden. Ausführliche Informationen zum Inhalt der Use Cases gibt es in Kapitel 5.2

4.3 Versuchsaufbau: Die vier Kontextstufen

4.3.1 Stufe 1 - Baseline: kein UI-Kontext

4.3.2 Stufe 2 - Automatischer Accessibility-Snapshot

4.3.3 Stufe 3 - Automatisch generierte UI-Map

4.3.4 Stufe 4 - Manuell erstellte UI-Map

4.3.5 Bonus-Stufe 5 - Self-Improvement-Loop

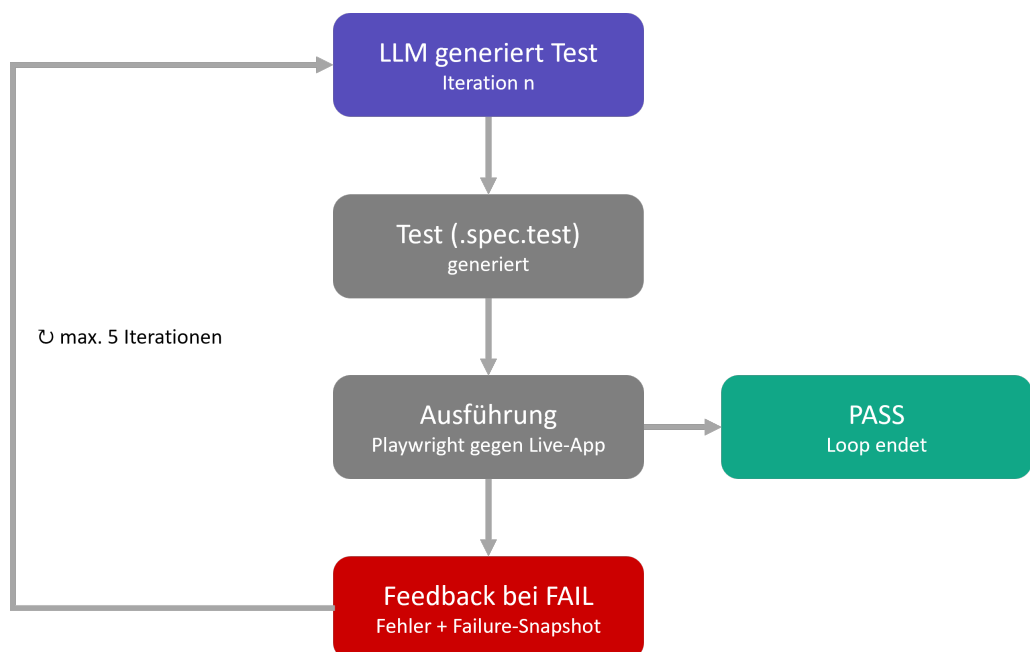


Abbildung 5: Self-Improvement-Loop der Bonus-Stufe 5: Fehlermeldung und Failure-Snapshot fließen in den nächsten Generierungsprompt zurück (max. 5 Iterationen)

4.4 LLM-Konfiguration (Modell, Temperatur, Prompt-Struktur)

4.5 Bewertungskriterien

5 Implementierung der Evaluationsumgebung

5.1 Demo-WebGIS-Anwendung

Die Demo-WebGIS-Anwendung stellt eine realistische, aber kontrollierte Testumgebung für die Evaluation bereit. Als Single-Page-Webanwendung umgesetzt, dient sie der weltweiten Erkundung von Wetterdaten und -vorhersagen. Die Anwendung ist unter <https://nils-gb156.github.io/ba-webgis-llm-e2e/> erreichbar und kann auch lokal gestartet werden. Der Quellcode ist auf GitHub unter <https://github.com/nils-gb156/ba-webgis-llm-e2e> verfügbar.

Abbildung 6 zeigt die Oberfläche der Anwendung. Den Mittelpunkt bildet die Karte. Auf der linken Seite befinden sich die Layersteuerung und die Legende, die über die Buttons in der Werkzeugleiste unten auch optional ein- und ausgeblendet werden können. Die Layersteuerung ermöglicht das Umschalten zwischen Hintergrundkarten sowie das Ein- und Ausblenden von Layern. Die Legende darunter zeigt die Symbolerklärungen der jeweils aktiven Layer.

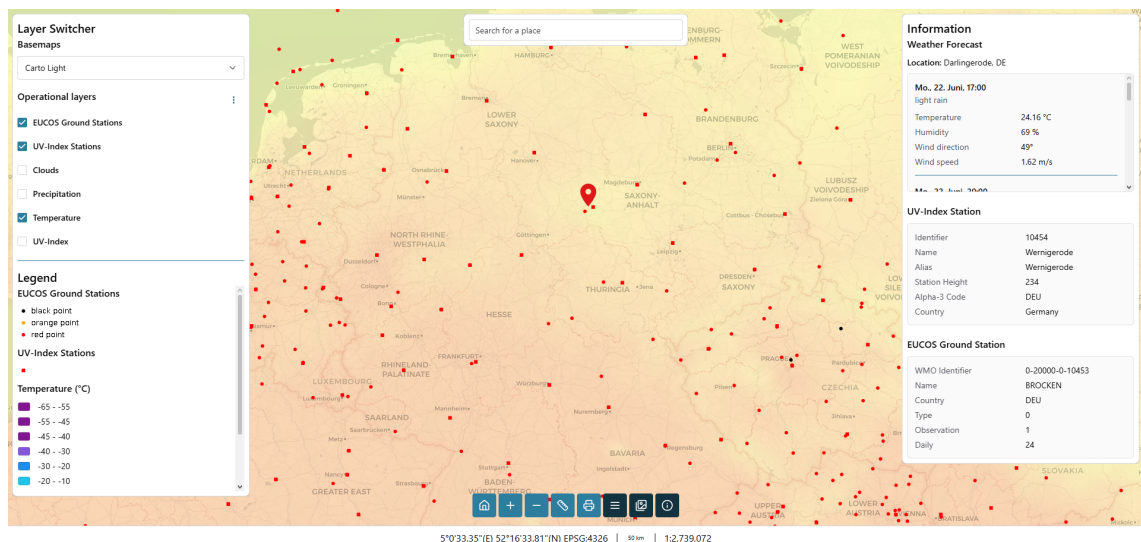


Abbildung 6: Oberfläche der Demo-WebGIS-Anwendung mit geklickter Koordinate

Die Anwendung bietet drei verschiedene Hintergrundkarten: Carto Light (Standard), Carto Dark und OpenStreetMap. Initial sichtbar sind zwei Punktlayer des Deutschen Wetterdienstes (DWD) für EUCOS-Bodenstationen und UV-Index-Stationen. Außerdem ist noch ein flächendeckender Layer der aktuellen Temperatur von OpenWeatherMap sichtbar. Über die Layersteuerung können weitere flächendeckende Layer für Niederschlag und

Wolkenbedeckung ebenfalls von OpenWeatherMap sowie ein europaweiter UV-Index-Layer des DWD eingeblendet werden.

Ein Klick auf die Karte öffnet ein Infopanel, das den Ort und die Wettervorhersage für die nächsten drei Tage im Drei-Stunden-Takt anzeigt. Die insgesamt 24 Einträge werden über die OpenWeatherMap API abgerufen und enthalten jeweils eine kurze Beschreibung des Wetters, Temperatur, Luftfeuchtigkeit sowie Windrichtung und -geschwindigkeit. Wird zusätzlich auf eine Bodenstation oder UV-Index-Station geklickt, werden die Stationsinformationen aus der WMS-GetFeatureInfo-Anfrage unterhalb der Wettervorhersage angezeigt.

Oben in der Mitte der Karte befindet sich ein Feld für eine Freitextsuche, die über die Nominatim-Geocoding-API nach Orten sucht. Bei Auswahl eines Suchergebnisses wird die Karte auf den Ort zentriert und das Infopanel mit der Wettervorhersage angezeigt.

Über die Werkzeugleiste lässt sich außerdem zur initialen Ausdehnung zurücknavigieren sowie der Kartenausschnitt vergrößern oder verkleinern. Zwei weitere Buttons rechts daneben öffnen verschiebbare Floating-Panels für Mess- und Druckfunktion. Das Messwerkzeug ermöglicht das direkte Messen von Linien und Flächen in der Karte. Mit der Druckfunktion kann der aktuelle Kartenausschnitt mit allen aktiven Layern als Portable Document Format (PDF) oder Portable Network Graphics (PNG) exportiert werden. Im Footer werden die aktuellen Kartenkoordinaten in World Geodetic System 1984 (WGS84) sowie eine Maßstabsleiste und der Maßstab angezeigt.

Die Anwendung wurde mit Open Pioneer Trails entwickelt und basiert auf OpenLayers, React, Chakra UI und TypeScript. Sie nutzt die verfügbaren OPT-Pakete für Layersteuerung, Kartennavigation, Legende, Messung, Druck und Koordinatenanzeige. Die Kartenprojektion ist European Petroleum Survey Group (EPSG):3857, die Koordinatenanzeige im Footer transformiert diese zu EPSG:4326.

5.2 Use-Case-Sammlung

Insgesamt repräsentieren 10 Use-Cases typische Nutzerinteraktionen mit der Demo-WebGIS-Anwendung. Die daraus generierten Tests sollen jeweils ihren Use-Case vollständig abdecken. Um sie auch maschinell verarbeitbar zu machen und eine einheitliche Struktur zu gewährleisten, werden sie im JavaScript Object Notation (JSON)-Format erfasst. Jeder Use-Case ist dabei wie folgt aufgebaut: `id`, `title`, `description`, `preconditions[]`, `steps[]`, `expected_result[]`, `complexity`. Titel und Beschreibung geben an, welche Nutzerinteraktion der Use-Case beschreibt. Die

Vorbedingungen enthalten Informationen, die vor der Ausführung erfüllt sein müssen. Die Schritte beschreiben die durchzuführenden Nutzerinteraktionen, und die erwarteten Ergebnisse definieren, welcher Zustand nach der Ausführung eintreten soll.

Zusätzlich werden die Use-Cases in drei Komplexitätsstufen eingeteilt: easy, medium und hard. Die Einstufung richtet sich dabei nicht nur nach der Anzahl der Schritte, sondern vor allem nach der Art und Tiefe der Interaktionen. Einfache Use-Cases testen einzelne Schaltflächen oder Sichtbarkeiten, während komplexere Use-Cases asynchrone Prozesse, Canvas-Interaktionen oder das Zusammenspiel mehrerer Komponenten erfordern. Der komplexeste Use-Case ist UC-10, da er Layer-Sichtbarkeiten, Geocoder-Suche, Kartennavigation und Wettervorhersage in einem einzigen Ablauf kombiniert.

Eine Übersicht aller Use-Cases ist in Tabelle 1 dargestellt. Da es sich hier um eine Demo-Anwendung handelt, sind die Use-Cases bewusst begrenzt. Reale WebGIS-Anwendungen können in der Praxis deutlich umfangreichere Nutzerinteraktionen erfordern.

Tabelle 1: Übersicht der Use Cases mit Komplexität und getesteter Kernfunktion

ID	Titel	Komplexität	Kernfunktion
1	Layer Switcher ein-/ausblenden	easy	Panel-Sichtbarkeit
2	Hintergrundkarte wechseln	easy	Basemap-Wechsel
3	Zoom-Buttons verwenden	easy	Zoom-Steuerung
4	UV-Index-Overlay aktivieren	medium	Layer-Rendering
5	Niederschlags-Overlay + Legende	medium	Layer + Legende
6	Wettervorhersage per Kartenklick	hard	Kartenklick-Abfrage
7	UVI-Station abfragen	hard	WMS GetFeatureInfo
8	Distanz messen	hard	Messwerkzeug
9	Karte als PNG exportieren	hard	Druckfunktion
10	Layer konfigurieren, Suche + Vorhersage	hard	Geocoder + Workflow

5.3 Manuell erstellte Referenztests

Für jeden Use Case wurde manuell ein Playwright-Test erstellt. Die Erstellung erfolgte vor der LLM-Generierung, um eine Beeinflussung zu vermeiden. Wären die Referenztests nach den generierten Tests geschrieben, bestünde die Gefahr, sich unbewusst an deren Lösungen zu orientieren. Die Referenztests haben zwei Funktionen. Zum einen dienen sie als fachlich korrekte Referenzimplementierung, die zeigt, wie ein Use Case

sauber umgesetzt aussieht. Zum anderen bilden sie den Vergleichsmaßstab für die LLM-generierten Tests in der Evaluation (Kapitel 6).

Die Tests sind mit Playwright in TypeScript umgesetzt. Pro Use Case gibt es eine eigene Spec-Datei nach dem Namensschema `uc-01-*.spec.ts` bis `uc-10-*.spec.ts`. Jeder Test folgt demselben Aufbau, der die Struktur des Use Cases widerspiegelt: Zuerst wird geprüft, ob die Vorbedingungen erfüllt sind, dann werden die Schritte ausgeführt und zuletzt die erwarteten Ergebnisse überprüft. Listing 1 zeigt diesen Aufbau am Beispiel von Use Case 4.

Bevor ein Test mit der Webanwendung interagiert, muss sichergestellt sein, dass die Anwendung vollständig geladen ist. Statt einer festen Wartezeit (Timeout), die unzuverlässig ist, wird mit `page.waitForFunction(() => globalThis.__openPioneerMap !== null)` gewartet, bis das Kartenmodell verfügbar ist. Sobald das Objekt existiert, ist die Karte einsatzbereit. Der Test wartet damit auf ein deterministisches Signal und nicht auf eine geschätzte Dauer. Dieses Muster ist in jedem Test enthalten.

Listing 1: UseCase-4: UV-Index Layer einblenden und prüfen, ob es auf der Karte angezeigt wird (Playwright-Test)

```

1 import { test, expect } from "@playwright/test";
2 import { isLayerRendered } from "../../map-model-helpers";
3
4 const UVI_LAYER_TITLE = "UV-Index";
5
6 test("UC-4: activate the UV-Index overlay and verify it is rendered on the map", async
    ({ page }) => {
7     await page.goto("http://localhost:5173/ba-webgis-llm-e2e/");
8
9     const map = page.getByTestId("map-container");
10    const layerSwitcher = page.getByTestId("layer-switcher");
11    // The TOC renders each layer with a checkbox whose accessible name is the layer
12    // title. `exact` avoids also matching the "UV-Index Stations" layer.
13    const uviToggle = layerSwitcher.getByRole("checkbox", { name: UVI_LAYER_TITLE,
14        exact: true });
15    const uviLabel = layerSwitcher.getByText(UVI_LAYER_TITLE, { exact: true });
16
17    // Preconditions
18    await expect(map).toBeVisible();
19    await page.waitForFunction(
20        () => (globalThis as { __openPioneerMap?: unknown }).__openPioneerMap !== null
21    );
22    await expect(layerSwitcher).toBeVisible();
23    await expect(uviToggle).not.toBeChecked();
24    expect(await isLayerRendered(page, UVI_LAYER_TITLE)).toBe(false);
25
26    // Steps
27    await uviLabel.click();

```

```

27 // Expected result
28 await expect(uviToggle).toBeChecked();
29 await expect.poll(() => isLayerRendered(page, UVI_LAYER_TITLE)).toBe(true);
30 });

```

Die Locator-Strategie basiert auf der in Kapitel 2 beschriebenen Hierarchie und nutzt bevorzugt `getByTestId`, dann `getByRole` und dann `getByText`. CSS-Selektoren und XPath werden vermieden, da sie an die DOM-Struktur gebunden und damit fragil sind. Die Selektoren müssen zudem bewusst präzise gewählt werden, da es sonst zu Fehlzugriffen kommen kann. Zum Beispiel wird in Use Case 4 der UV-Index Layer über `getByRole("checkbox", { name: "UV-Index", exact: true })` angesprochen. Die Option `exact` ist hier notwendig, weil sonst auch der Layer "ÜV-Index Stations" mitgetroffen werden würde, was im Rahmen des Use Cases falsch wäre. Eine Ausnahme ist Use Case 9. Dort werden zwei Elemente über CSS-Klassen angesprochen, weil die betreffenden internen Elemente der Druckfunktion von Open Pioneer Trails keine eigene Test-ID besitzen.

Da der Karteninhalt auf einem Canvas gerendert wird (siehe Kapitel 2.4), gibt es dort keine DOM-Elemente. Klicks auf der Karte erfolgen daher über `page.mouse.click(x, y)` mit Pixelkoordinaten, die aus der `boundingBox()` des Kartencontainers berechnet werden. Dabei gibt es einen wichtigen Unterschied zwischen zwei Arten von Kartenklicks. Bei Use Case 6 genügt ein Klick auf eine beliebige Position, etwa die Kartenmitte. Bei Use Case 7 hingegen muss ein konkreter Punkt getroffen werden. Dafür wird die bekannte Koordinate (EPSG:3857) über die OpenLayers-Funktion `getPixelFromCoordinate()` in eine Canvas-Pixelposition umgerechnet, bevor geklickt wird. Das gezielte Treffen eines Features ist damit deutlich aufwendiger als ein beliebiger Klick in die Karte.

Der Kartenzustand, dazu gehören aktive Layer, Zoomstufe, Zentrum und Highlight, ist nicht über das DOM auslesbar. In der Datei `map-model-helpers.ts` sind Funktionen, die auf das `MapModel` zugreifen. Sie sind bewusst ausgelagert, da sie unabhängig von der Webanwendung sind und auch in anderen Open Pioneer Trails-Anwendungen wiederverwendet werden können. Passend für die Use Cases stellt die Datei aktuell die Funktionen `getActiveBaseLayerTitle()`, `isLayerRendered()`, `getMapZoomLevel()`, `getMapCenter()` und `getHighlightedCoordinate()` bereit. Die Funktionen müssen nur eingebunden werden und können dann direkt im Test verwendet werden.

Zum Beispiel geht es in Use Case 4 um das Aktivieren des UV-Index Layers. Dies kann nun relativ einfach auf zwei Ebenen geprüft werden. Erstens auf DOM-Ebene, dass der Toggle in der Layersteuerung den Zustand `aktiviert` (`toBeChecked`) hat. Zweitens auf Modell-Ebene, dass der Layer auch tatsächlich gerendert wird (`isLayerRendered`). Nur weil

ein Layer in der Layersteuerung aktiviert wurde, bedeutet das noch nicht, dass die Kacheln auch wirklich auf der Karte erscheinen. Die Prüfung auf Modell-Ebene nutzt `expect.poll()`, da die Kacheln asynchron über das Netz geladen werden. Ein reines `expect` ist hier nicht ausreichend, da es nur einen festen Wert prüft und nicht wiederholt nachfragt. Mit `expect.poll()` wird die Funktion wiederholt aufgerufen, bis der Layer als gerendert gilt oder ein Timeout erreicht wird. Somit wird sichergestellt, dass die Kacheln geladen sind, bevor der Test fortschreitet. Konkret sieht das dann so aus: `await expect.poll(() => isLayerRendered(page, UVI_LAYER_TITLE)).toBe(true)`.

In Use Case 7 wird zusätzlich überprüft, dass tatsächlich ein WMS-GetFeatureInfo-Request an den DWD-Dienst gesendet wird. Dazu werden mit `page.on("request", ...)` die ausgehenden Requests beobachtet.

6 Durchführung und Evaluation

6.1 LLM-gestützter Generierungsworkflow

6.2 Ergebnisse je Kontextstufe

6.3 Vergleich der Kontextstufen

6.4 Vergleich mit manuell erstellten Tests

6.5 GIS-spezifische Besonderheiten

7 Diskussion

7.1 Interpretation der Ergebnisse

7.2 Limitationen

7.3 Handlungsempfehlungen

8 Fazit und Ausblick

Anhang

Anhang 1: Beispielanhang

Dieser Abschnitt dient nur dazu zu demonstrieren, wie ein Anhang aufgebaut sein kann.

Anhang 1.1: Weitere Gliederungsebene

Auch eine zweite Gliederungsebene ist möglich.

Anhang 2: Bilder

Auch mit Bildern. Diese tauchen nicht im Abbildungsverzeichnis auf.

Name	Änderungsdatum	Typ	Größe
 abbildungen	29.08.2013 01:25	Dateiordner	
 kapitel	29.08.2013 00:55	Dateiordner	
 literatur	31.08.2013 18:17	Dateiordner	
 skripte	01.09.2013 00:10	Dateiordner	
 compile.bat	31.08.2013 20:11	Windows-Batchda...	1 KB
 thesis_main.tex	01.09.2013 00:25	LaTeX Document	5 KB

Abbildung 7: Beispielbild

Literaturverzeichnis

- 52 North GmbH, Arne Vogt* (2025): Open Pioneer Trails: An Open-Source Framework for Map-Oriented Web Apps, en-US, o. O., 2025-11, URL: <https://blog.52north.org/2025/11/10/open-pioneer-trails/>
- 52 North GmbH, Matthes Rieke Managing Director* (2026): Conquering new frontiers in web mapping application development, en-US, o. O., 2026-06, URL: <https://52north.org/software/software-components/open-pioneer-trails/>
- Alian, Parsa, Nashid, Noor, Shahbandeh, Mobina, Shabani, Taha, Mesbah, Ali* (2024): Feature-Driven End-To-End Test Generation, Version Number: 2, o. O., 2024, URL: <https://arxiv.org/abs/2408.01894>
- Brown, Tom B., Mann, Benjamin, Ryder, Nick, Subbiah, Melanie, Kaplan, Jared, Dhariwal, Prafulla, Neelakantan, Arvind, Shyam, Pranav, Sastry, Girish, Askell, Amanda, Agarwal, Sandhini, Herbert-Voss, Ariel, Krueger, Gretchen, Henighan, Tom, Child, Rewon, Ramesh, Aditya, Ziegler, Daniel M., Wu, Jeffrey, Winter, Clemens, Hesse, Christopher, Chen, Mark, Sigler, Eric, Litwin, Mateusz, Gray, Scott, Chess, Benjamin, Clark, Jack, Berner, Christopher, McCandlish, Sam, Radford, Alec, Sutskever, Ilya, Amodei, Dario* (2020): Language Models are Few-Shot Learners, Version Number: 4, o. O., 2020, URL: <https://arxiv.org/abs/2005.14165>
- Celik, Arda, Mahmoud, Qusay H.* (2025): A Review of Large Language Models for Automated Test Case Generation, en, in: Machine Learning and Knowledge Extraction, 7 (2025), Nr. 3, S. 97
- Chakra Systems* (2026): Chakra Factory | Chakra UI, en, o. O., 2026-06, URL: <https://chakra-ui.com/docs/styling/chakra-factory>
- Ferreira, Margarida, Viegas, Luís, Faria, João Pascoal, Lima, Bruno* (2025): Acceptance Test Generation with Large Language Models: An Industrial Case Study, in: 2025 IEEE/ACM International Conference on Automation of Software Test (AST), o. O.: IEEE, 2025-04, S. 1–11
- Hou, Xinyi, Zhao, Yanjie, Liu, Yue, Yang, Zhou, Wang, Kailong, Li, Li, Luo, Xiapu, Lo, David, Grundy, John, Wang, Haoyu* (2024): Large Language Models for Software Engineering: A Systematic Literature Review, in: ACM Trans. Softw. Eng. Methodol. 33 (2024), Nr. 8, 220:1–220:79
- Microsoft* (2026): Fast and reliable end-to-end testing for modern web apps | Playwright, en, Dokumentation, o. O., 2026-06, URL: <https://playwright.dev/>

Open Pioneer (2026): Open Pioneer GitHub, en, o. O., 2026-06, URL: <https://github.com/open-pioneer>

OpenLayers (Institution) (2026): Class: CanvasImmediateRenderer (OpenLayers v10.9.0 API), o. O., 2026-06, URL: https://openlayers.org/en/latest/apidoc/module-ol_render_canvas_Immediate-CanvasImmediateRenderer.html

Plein, Laura, Ouédraogo, Wendkûni C., Klein, Jacques, Bissyandé, Tegawendé F. (2023): Automatic Generation of Test Cases based on Bug Reports: a Feasibility Study with Large Language Models, Version Number: 1, o. O., 2023, URL: <https://arxiv.org/abs/2310.06320>

Poenisch, Marie (2022): Die Testpyramide, de, o. O., 2022-08, URL: <https://www.openknowledge.de/blog/die-testpyramide>

Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Lukasz, Polosukhin, Illia (2017): Attention Is All You Need, Version Number: 7, o. O., 2017, URL: <https://arxiv.org/abs/1706.03762>